🤖 SO-101 Hand Mimicking Project: Kompletny Przewodnik Krok-po-Kroku

Autor: Robotics Engineer
Data: 09 marca 2026
Robot: SO-101 Arm (LeRobot compatible)
Goal: Mimikowanie ruchu ręki z kamery → symulacja Gazebo
Inspiracja: myCobot YOLOv8 Hand Tracking Project

📑 Spis Treści

1. Przegląd projektu i architektura systemu
2. SO-101: Specyfikacja i źródła modeli CAD
3. Setup środowiska (ROS 2 Jazzy + Gazebo Harmonic)
4. Import SO-101 do Gazebo (URDF/Xacro)
5. Proste komendy ruchu przez ROS 2
6. OpenCV + MediaPipe: Detekcja ręki
7. Mapowanie trajektorii ręki na joint space robota
8. Przeniesienie ruchu do symulacji Gazebo
9. Testing i debugging
10. Następne kroki: Shape sorting project

---

1. Przegląd Projektu i Architektura Systemu

1.1 Cel Projektu

Mission Statement:
Stworzenie systemu vision-based teleoperation, gdzie SO-101 w Gazebo replikuje ruch ludzkiej ręki wykrywany przez kamerę USB. Projekt stanowi fundament pod przyszłe aplikacje manipulation (shape sorting, pick-and-place).

1.2 Architektura High-Level

Figure 1: Pipeline projektu: kamera → detekcja → mapowanie → symulacja

Komponenty systemu:

- **Vision Module:** Kamera USB + MediaPipe Hand Tracking
- **Mapping Module:** Transformacja hand landmarks → joint angles SO-101
- **Control Module:** ROS 2 nodes publikujące joint commands
- **Simulation Module:** Gazebo Harmonic z modelem SO-101 URDF

1.3 Tech Stack

Component	Technology
OS	Ubuntu 24.04 LTS
Middleware	ROS 2 Jazzy
Simulator	Gazebo Harmonic
Computer Vision	OpenCV 4.10 + MediaPipe 0.10
Programming	Python 3.12 (vision) + C++ (optional control)
Robot Description	URDF/Xacro

Table 1: Stack technologiczny projektu

1.4 Timeline i Milestones

Szacowany czas realizacji: 35-45 godzin (3-4 tygodnie part-time)

- **Week 1:** Setup + SO-101 model w Gazebo (10-12h)
- **Week 2:** ROS 2 control + vision pipeline (12-15h)
- **Week 3:** Mapping + integration (10-12h)
- **Week 4:** Testing, tuning, dokumentacja (3-6h)

---

2. SO-101: Specyfikacja i Źródła Modeli CAD

2.1 SO-101 Specifications

SO-101 (Standard Open Arm 101) - Official LeRobot Platform

Parameter	Value
DOF (Degrees of Freedom)	5 (joint 0-4)
Reach	~400mm
Payload	250g
Joint Types	5× Revolute (Dynamixel servos)
End-Effector	1-DOF gripper (optional for phase 1)
Weight	~1.2kg
Power	12V DC

Table 2: SO-101 technical specifications

Joint Configuration:

- **Joint 0:** Base rotation (yaw) - range: -180° to +180°
- **Joint 1:** Shoulder pitch - range: -90° to +90°
- **Joint 2:** Elbow pitch - range: -135° to +135°
- **Joint 3:** Wrist pitch - range: -90° to +90°
- **Joint 4:** Wrist roll - range: -180° to +180°

UWAGA: W naszym projekcie użyjemy **tylko Joint 1 i Joint 2** (shoulder + elbow) do naśladowania Y i Z pozycji ręki, zgodnie z Twoją specyfikacją.

2.2 Gdzie Pobrać Model CAD SO-101

Opcja 1: Official LeRobot Repository (RECOMMENDED)

GitHub Repository:

Sklonuj oficjalne repozytorium LeRobot

git clone https://github.com/huggingface/lerobot.git
cd lerobot

Przejdź do katalogu z modelami SO-100 (SO-101 based on SO-100)

cd robot_descriptions/so100

Zawartość:

- `meshes/` - STL/OBJ files dla visual i collision
- `urdf/` - URDF description files
- `config/` - Joint limits, controllers config

Link bezpośredni:
https://github.com/huggingface/lerobot/tree/main/lerobot/configs/robot/so100.yaml

Opcja 2: Community ROS 2 Package

GitHub: brukg/SO-100-arm

Alternatywne repozytorium z gotowym ROS 2 package

git clone https://github.com/brukg/SO-100-arm.git
cd SO-100-arm

Zawiera:

- Kompletny ROS 2 package z URDF
- Gazebo launch files
- Example controllers

- Dokumentacja setup

Link: https://github.com/brukg/SO-100-arm

Opcja 3: Onshape CAD (jeśli chcesz modyfikować model)

Onshape Public Document:

- Wyszukaj "SO-100 Arm" w Onshape Community
- Fork document do własnego workspace
- Export jako STEP/STL dla Gazebo

Link przykładowy: https://cad.onshape.com/documents/... (search: "Standard Open Arm 100")

2.3 Weryfikacja Pobranych Plików

Sprawdź strukturę plików:

Po sklonowaniu lerobot repo

tree lerobot/configs/robot/

Powinno pokazać:

├── so100.yaml

├── so100_urdf/

│ ├── so100.urdf

│ ├── so100.xacro

│ └── meshes/

│ ├── base_link.stl

│ ├── link1.stl

│ ├── link2.stl

│ ├── link3.stl

│ └── link4.stl

UWAGA: Jeśli pliki mesh są w formacie OBJ/DAE, skonwertuj do STL dla lepszej kompatybilności z Gazebo:

Install meshlabserver (do konwersji)

sudo apt install meshlab

Konwersja OBJ → STL

meshlabserver -i input.obj -o output.stl

---

3. Setup Środowiska: ROS 2 Jazzy + Gazebo Harmonic

3.1 System Requirements

Hardware minimum:

- CPU: Intel i5 8th gen / AMD Ryzen 5 3600
- RAM: 16GB (8GB for OS + 8GB for Gazebo)
- GPU: Zintegrowana Intel/AMD (wystarczy!)
- Dysk: 30GB free space (SSD rekomendowany)
- Kamera: USB 2.0/3.0 webcam (720p minimum, 1080p better)

3.2 Instalacja Ubuntu 24.04 LTS

Jeśli jeszcze nie masz Ubuntu 24.04:

Download Ubuntu 24.04 LTS

https://ubuntu.com/download/desktop

Bootable USB tool: Rufus (Windows) or Etcher (Linux/Mac)

Install alongside Windows (dual boot) lub jako główny OS

3.3 Instalacja ROS 2 Jazzy

Official installation method:

Set locale

sudo apt update && sudo apt install locales
sudo locale-gen en_US en_US.UTF-8
sudo update-locale LC_ALL=en_US.UTF-8 LANG=en_US.UTF-8
export LANG=en_US.UTF-8

Setup sources

sudo apt install software-properties-common
sudo add-apt-repository universe
sudo apt update && sudo apt install curl -y
sudo curl -sSL https://raw.githubusercontent.com/ros/rosdistro/master/ros.key
-o /usr/share/keyrings/ros-archive-keyring.gpg
echo "deb [arch=$(dpkg --print-architecture)
signed-by=/usr/share/keyrings/ros-archive-keyring.gpg]
http://packages.ros.org/ros2/ubuntu $(. /etc/os-release && echo
$UBUNTU_CODENAME) main"
| sudo tee /etc/apt/sources.list.d/ros2.list > /dev/null

Install ROS 2 Jazzy Desktop (full version)

sudo apt update
sudo apt upgrade
sudo apt install ros-jazzy-desktop

Install development tools

sudo apt install ros-dev-tools

Weryfikacja instalacji:

Source ROS 2

source /opt/ros/jazzy/setup.bash

Test

ros2 run demo_nodes_cpp talker

W drugim terminalu:

ros2 run demo_nodes_py listener

Powinno działać!

Automatyczne source przy starcie terminala:

echo "source /opt/ros/jazzy/setup.bash" >> ~/.bashrc
source ~/.bashrc

3.4 Instalacja Gazebo Harmonic

Gazebo Harmonic (formerly Ignition Fortress) - najnowsza wersja:

Add Gazebo repository

sudo wget https://packages.osrfoundation.org/gazebo.gpg
-O /usr/share/keyrings/pkgs-osrf-archive-keyring.gpg
echo "deb [arch=$(dpkg --print-architecture)
signed-by=/usr/share/keyrings/pkgs-osrf-archive-keyring.gpg]
http://packages.osrfoundation.org/gazebo/ubuntu-stable
$(lsb_release -cs) main"
| sudo tee /etc/apt/sources.list.d/gazebo-stable.list > /dev/null

Install Gazebo Harmonic

sudo apt update
sudo apt install gz-harmonic

Install ROS 2 - Gazebo bridge

sudo apt install ros-jazzy-ros-gz

Weryfikacja Gazebo:

Uruchom Gazebo GUI

gz sim

Powinno otworzyć się okno Gazebo z pustym światem

3.5 Instalacja OpenCV i MediaPipe

OpenCV 4.10 (ROS 2 compatible):

Install via apt (recommended for ROS 2 integration)

sudo apt install python3-opencv
sudo apt install ros-jazzy-cv-bridge ros-jazzy-vision-opencv

Weryfikacja

python3 -c "import cv2; print(cv2.**version**)"

Output: 4.10.x

MediaPipe (dla hand tracking):

Install via pip

pip3 install mediapipe==0.10.14

Weryfikacja

python3 -c "import mediapipe as mp; print(mp.**version**)"

Output: 0.10.14

3.6 Dodatkowe Narzędzia

RViz2 (visualization), Colcon (build tool), Git:

RViz2 (included in ros-jazzy-desktop)

sudo apt install ros-jazzy-rviz2

Colcon build tool

sudo apt install python3-colcon-common-extensions

Git, VSCode, etc.

sudo apt install git gitg
sudo snap install code --classic

Terminator (better terminal)

sudo apt install terminator

3.7 Workspace Setup

Stwórz ROS 2 workspace dla projektu:

Create workspace

mkdir -p ~/ros2_ws/src
cd ~/ros2_ws

Build empty workspace (initialize)

colcon build
source install/setup.bash

Add to .bashrc for auto-sourcing

echo "source ~/ros2_ws/install/setup.bash" >> ~/.bashrc

---

4. Import SO-101 do Gazebo: URDF/Xacro

4.1 Utworzenie ROS 2 Package dla SO-101

Generate package structure:

cd ~/ros2_ws/src

Create package

ros2 pkg create so101_description
--build-type ament_cmake
--dependencies urdf xacro

cd so101_description
mkdir -p urdf meshes launch config

4.2 Pobranie i Adaptacja URDF

Sklonuj oficjalne pliki SO-100 (base dla SO-101):

cd ~/ros2_ws/src
git clone https://github.com/brukg/SO-100-arm.git

Skopiuj niezbędne pliki do naszego package

cp -r SO-100-arm/urdf/* so101_description/urdf/
cp -r SO-100-arm/meshes/* so101_description/meshes/

Struktura plików po skopiowaniu:

```
so101_description/
├── CMakeLists.txt
├── package.xml
├── urdf/
│  ├── so101.urdf.xacro
│  ├── so101_gazebo.xacro
│  └── materials.xacro
├── meshes/
│  ├── base_link.stl
│  ├── link1.stl
│  ├── link2.stl
│  ├── link3.stl
│  ├── link4.stl
│  └── gripper.stl (opcjonalny)
```

```
├── launch/
└── config/
```

4.3 Podstawowy URDF SO-101 (Simplified)

Stwórz plik `urdf/so101_minimal.urdf.xacro`:

UWAGA: Ten URDF skupia się na **joint_1 i joint_2**, które będziemy kontrolować na podstawie pozycji ręki (Y i Z).

4.4 Gazebo Plugin dla Kontroli Jointów

Stwórz plik `urdf/so101_gazebo.xacro`:

$(find so101_description)/config/so101_controllers.yaml Gazebo/Grey Gazebo/Blue Gazebo/Blue Gazebo/Grey Gazebo/Grey Gazebo/Red

4.5 Controller Configuration

Stwórz plik `config/so101_controllers.yaml`:

controller_manager:
ros__parameters:
update_rate: 100 # Hz

```
joint_state_broadcaster:
  type: joint_state_broadcaster/JointStateBroadcaster

position_controller:
  type: position_controllers/JointGroupPositionController
```

position_controller:
ros__parameters:
joints:
- joint_0
- joint_1
- joint_2
- joint_3
- joint_4

```
command_interfaces:
  - position

state_interfaces:
  - position
  - velocity
```

4.6 Launch File dla Gazebo

Stwórz `launch/gazebo_so101.launch.py`:

```
import os
from ament_index_python.packages import
get_package_share_directory
from launch import LaunchDescription
from launch.actions import DeclareLaunchArgument,
IncludeLaunchDescription
from launch.launch_description_sources import
PythonLaunchDescriptionSource
from launch.substitutions import LaunchConfiguration
from launch_ros.actions import Node
import xacro

def generate_launch_description():
# Paths
pkg_so101_description =
get_package_share_directory('so101_description')
```

```
# URDF file path
urdf_file = os.path.join(pkg_so101_description, 'urdf', 'so101_minimal.urdf.x
robot_description = xacro.process_file(urdf_file).toxml()

# Gazebo launch
gazebo = IncludeLaunchDescription(
    PythonLaunchDescriptionSource([
        os.path.join(get_package_share_directory('ros_gz_sim'),
                     'launch', 'gz_sim.launch.py')
    ]),
```

```
    launch_arguments={'gz_args': '-r empty.sdf'}.items()
)

# Robot State Publisher
robot_state_publisher = Node(
    package='robot_state_publisher',
    executable='robot_state_publisher',
    output='screen',
    parameters=[{'robot_description': robot_description}]
)

# Spawn entity in Gazebo
spawn_entity = Node(
    package='ros_gz_sim',
    executable='create',
    arguments=[
        '-name', 'so101',
        '-topic', 'robot_description',
        '-x', '0.0',
        '-y', '0.0',
        '-z', '0.01'
    ],
    output='screen'
)

# Joint State Broadcaster
joint_state_broadcaster = Node(
    package='controller_manager',
    executable='spawner',
    arguments=['joint_state_broadcaster'],
    output='screen'
)

# Position Controller
position_controller = Node(
    package='controller_manager',
    executable='spawner',
    arguments=['position_controller'],
```

```
    output='screen'
)

return LaunchDescription([
    gazebo,
    robot_state_publisher,
    spawn_entity,
    joint_state_broadcaster,
    position_controller
])
```

4.7 Update CMakeLists.txt i package.xml

CMakeLists.txt:

cmake_minimum_required(VERSION 3.8)
project(so101_description)

Find dependencies

find_package(ament_cmake REQUIRED)
find_package(urdf REQUIRED)
find_package(xacro REQUIRED)

Install directories

install(DIRECTORY
urdf
meshes
launch
config
DESTINATION share/${PROJECT_NAME}
)

ament_package()

package.xml:

so101_description 1.0.0 SO-101 robot arm description package Your Name MIT ament_cmake urdf xacro robot_state_publisher ros_gz_sim controller_manager joint_state_broadcaster position_controllers ament_cmake

4.8 Build i Test w Gazebo

Build package:

```
cd ~/ros2_ws
colcon build --packages-select so101_description
source install/setup.bash
```

Uruchom symulację:

```
ros2 launch so101_description gazebo_so101.launch.py
```

Powinno otworzyć się Gazebo z modelem SO-101!

Weryfikacja w drugim terminalu:

Sprawdź dostępne topiki

```
ros2 topic list
```

Powinny być:

/joint_states

/position_controller/commands

/robot_description

/tf

/tf_static

Sprawdź joint states

ros2 topic echo /joint_states

---

5. Proste Komendy Ruchu przez ROS 2

5.1 Manual Joint Control via CLI

Publikuj pozycje jointów bezpośrednio:

Format: ros2 topic pub <topic> <msg_type> "<data>"

Move joint_1 to 45 degrees (0.785 rad)

ros2 topic pub --once /position_controller/commands
std_msgs/msg/Float64MultiArray
"{data: [0.0, 0.785, 0.0, 0.0, 0.0]}"

Move joint_2 to -30 degrees (-0.524 rad)

ros2 topic pub --once /position_controller/commands
std_msgs/msg/Float64MultiArray
"{data: [0.0, 0.0, -0.524, 0.0, 0.0]}"

Move both joint_1 and joint_2 simultaneously

ros2 topic pub --once /position_controller/commands
std_msgs/msg/Float64MultiArray
"{data: [0.0, 0.785, -0.524, 0.0, 0.0]}"

Obserwuj ruch w Gazebo - powinieneś zobaczyć animację!

5.2 Python Node dla Kontroli Trajektorii

Stwórz nowy package `so101_control`:

cd ~/ros2_ws/src
ros2 pkg create so101_control
--build-type ament_python
--dependencies rclpy std_msgs

cd so101_control/so101_control

Stwórz plik `simple_trajectory.py`:

#!/usr/bin/env python3

import rclpy
from rclpy.node import Node
from std_msgs.msg import Float64MultiArray
import math
import time

class SimpleTrajectoryController(Node):
def **init**(self):
super().**init**('simple_trajectory_controller')

```
# Publisher for joint commands
self.joint_pub = self.create_publisher(
    Float64MultiArray,
    '/position_controller/commands',
    10
```

```
    )

    self.get_logger().info('Simple Trajectory Controller started')

def move_to_position(self, joint_positions):
    """
    Send joint position command
    Args:
        joint_positions: list of 5 floats [j0, j1, j2, j3, j4]
    """
    msg = Float64MultiArray()
    msg.data = joint_positions
    self.joint_pub.publish(msg)
    self.get_logger().info(f'Moving to: {joint_positions}')

def execute_wave_motion(self):
    """Execute a simple wave motion using joint_1 and joint_2"""
    self.get_logger().info('Starting wave motion...')

    # Define trajectory waypoints (5 joints: j0, j1, j2, j3, j4)
    waypoints = [
        [0.0, 0.0, 0.0, 0.0, 0.0],        # Home position
        [0.0, 0.5, -0.3, 0.0, 0.0],       # Up-right
        [0.0, 0.8, -0.6, 0.0, 0.0],       # More up
        [0.0, 0.5, -0.3, 0.0, 0.0],       # Back to mid
        [0.0, 0.0, 0.0, 0.0, 0.0],        # Home
        [0.0, -0.5, 0.3, 0.0, 0.0],       # Down-left
        [0.0, -0.8, 0.6, 0.0, 0.0],       # More down
        [0.0, -0.5, 0.3, 0.0, 0.0],       # Back to mid
        [0.0, 0.0, 0.0, 0.0, 0.0],        # Home
    ]

    # Execute trajectory
    for waypoint in waypoints:
        self.move_to_position(waypoint)
        time.sleep(1.5)  # Wait for motion to complete
```

```
    self.get_logger().info('Wave motion completed!')
```

def main(args=None):
rclpy.init(args=args)
controller = SimpleTrajectoryController()

```
# Execute wave motion
controller.execute_wave_motion()

# Keep node alive
rclpy.spin(controller)
controller.destroy_node()
rclpy.shutdown()
```

if **name** == '**main**':
main()

Update `setup.py`:

from setuptools import setup

package_name = 'so101_control'

setup(
name=package_name,
version='1.0.0',
packages=[package_name],
install_requires=['setuptools'],
zip_safe=True,
maintainer='Your Name',
maintainer_email='your@email.com',
description='SO-101 control nodes',
license='MIT',
tests_require=['pytest'],
entry_points={
'console_scripts': [
'simple_trajectory = so101_control.simple_trajectory:main',
],

```
},
)
```

Build i test:

```
cd ~/ros2_ws
colcon build --packages-select so101_control
source install/setup.bash
```

W pierwszym terminalu: uruchom Gazebo

```
ros2 launch so101_description gazebo_so101.launch.py
```

W drugim terminalu: uruchom trajectory controller

```
ros2 run so101_control simple_trajectory
```

Powinieneś zobaczyć SO-101 wykonujący "wave" motion w Gazebo!

---

6. OpenCV + MediaPipe: Detekcja Ręki

6.1 MediaPipe Hand Tracking - Teoria

MediaPipe Hand Landmarks:

MediaPipe wykrywa 21 punktów kluczowych (landmarks) na dłoni:

- **Landmark 0:** Wrist (nadgarstek)
- **Landmarks 1-4:** Thumb (kciuk)
- **Landmarks 5-8:** Index finger (palec wskazujący)

- **Landmarks 9-12:** Middle finger (palec środkowy)
- **Landmarks 13-16:** Ring finger (palec serdeczny)
- **Landmarks 17-20:** Pinky (mały palec)

Dla naszego projektu użyjemy:

- **Landmark 0 (Wrist):** Pozycja Y → Joint 1 (shoulder pitch)
- **Landmark 9 (Middle finger base):** Pozycja Z → Joint 2 (elbow pitch)

6.2 Standalone Hand Tracking Test

Stwórz plik `test_hand_tracking.py`:

```
#!/usr/bin/env python3

import cv2
import mediapipe as mp
import numpy as np

# Initialize MediaPipe Hands

mp_hands = mp.solutions.hands
mp_drawing = mp.solutions.drawing_utils
hands = mp_hands.Hands(
static_image_mode=False,
max_num_hands=1,
min_detection_confidence=0.7,
min_tracking_confidence=0.5
)

# Open webcam

cap = cv2.VideoCapture(0)
cap.set(cv2.CAP_PROP_FRAME_WIDTH, 640)
cap.set(cv2.CAP_PROP_FRAME_HEIGHT, 480)

print("Press 'q' to quit")
```

while cap.isOpened():
ret, frame = cap.read()
if not ret:
break

```
# Flip frame horizontally for mirror effect
frame = cv2.flip(frame, 1)

# Convert BGR to RGB
rgb_frame = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)

# Process frame
results = hands.process(rgb_frame)

# Draw landmarks
if results.multi_hand_landmarks:
    for hand_landmarks in results.multi_hand_landmarks:
        # Draw hand skeleton
        mp_drawing.draw_landmarks(
            frame,
            hand_landmarks,
            mp_hands.HAND_CONNECTIONS
        )

        # Extract wrist (landmark 0) and middle finger base (landmark 9)
        wrist = hand_landmarks.landmark[0]
        middle_base = hand_landmarks.landmark[9]

        # Get pixel coordinates
        h, w, _ = frame.shape
        wrist_x = int(wrist.x * w)
        wrist_y = int(wrist.y * h)
        middle_x = int(middle_base.x * w)
        middle_y = int(middle_base.y * h)

        # Draw circles on key points
        cv2.circle(frame, (wrist_x, wrist_y), 10, (0, 255, 0), -1)
        cv2.circle(frame, (middle_x, middle_y), 10, (255, 0, 0), -1)
```

```
        # Display normalized coordinates
        cv2.putText(frame, f"Wrist Y: {wrist.y:.3f}",
                    (10, 30), cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 255, 0), 2)
        cv2.putText(frame, f"Middle Z: {middle_base.y:.3f}",
                    (10, 60), cv2.FONT_HERSHEY_SIMPLEX, 0.7, (255, 0, 0), 2)

    # Show frame
    cv2.imshow('Hand Tracking Test', frame)

    if cv2.waitKey(1) & 0xFF == ord('q'):
        break
```

cap.release()
cv2.destroy AllWindows()
hands.close()

Uruchom test:

python3 test_hand_tracking.py

Pokaż rękę przed kamerą - powinieneś zobaczyć skeleton i współrzędne!

6.3 ROS 2 Node dla Hand Tracking

Stwórz `so101_control/hand_tracker.py`:

#!/usr/bin/env python3

import rclpy
from rclpy.node import Node
from std_msgs.msg import Float64MultiArray
from geometry_msgs.msg import Point
import cv2

import mediapipe as mp
import numpy as np

class HandTracker(Node):
def **init**(self):
super().**init**('hand_tracker')

```
    # Publishers
    self.hand_pose_pub = self.create_publisher(Point, '/hand_pose', 10)

    # MediaPipe setup
    self.mp_hands = mp.solutions.hands
    self.mp_drawing = mp.solutions.drawing_utils
    self.hands = self.mp_hands.Hands(
        static_image_mode=False,
        max_num_hands=1,
        min_detection_confidence=0.7,
        min_tracking_confidence=0.5
    )

    # Camera setup
    self.cap = cv2.VideoCapture(0)
    self.cap.set(cv2.CAP_PROP_FRAME_WIDTH, 640)
    self.cap.set(cv2.CAP_PROP_FRAME_HEIGHT, 480)

    # Timer for processing frames
    self.timer = self.create_timer(0.03, self.process_frame)  # ~30 FPS

    self.get_logger().info('Hand Tracker started')

def process_frame(self):
    ret, frame = self.cap.read()
    if not ret:
        return

    # Flip for mirror effect
    frame = cv2.flip(frame, 1)
    rgb_frame = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
```

```
    # Process with MediaPipe
    results = self.hands.process(rgb_frame)

    if results.multi_hand_landmarks:
        for hand_landmarks in results.multi_hand_landmarks:
            # Draw landmarks
            self.mp_drawing.draw_landmarks(
                frame,
                hand_landmarks,
                self.mp_hands.HAND_CONNECTIONS
            )

            # Extract key landmarks
            wrist = hand_landmarks.landmark[0]
            middle_base = hand_landmarks.landmark[9]

            # Publish hand pose (normalized 0-1)
            hand_pose = Point()
            hand_pose.x = wrist.x  # Not used in phase 1
            hand_pose.y = wrist.y  # Maps to joint_1
            hand_pose.z = middle_base.y  # Maps to joint_2
            self.hand_pose_pub.publish(hand_pose)

            # Visualization
            h, w, _ = frame.shape
            cv2.putText(frame, f"Y: {wrist.y:.2f} Z: {middle_base.y:.2f}",
                        (10, 30), cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 255, 0), 2)

    # Display frame
    cv2.imshow('Hand Tracker', frame)
    cv2.waitKey(1)

def destroy_node(self):
    self.cap.release()
    cv2.destroyAllWindows()
```

```
    self.hands.close()
    super().destroy_node()
```

def main(args=None):
rclpy.init(args=args)
hand_tracker = HandTracker()

```
try:
    rclpy.spin(hand_tracker)
except KeyboardInterrupt:
    pass
finally:
    hand_tracker.destroy_node()
    rclpy.shutdown()
```

if **name** == '**main**':
main()

Update `setup.py` (dodaj entry point):

entry_points={
'console_scripts': [
'simple_trajectory = so101_control.simple_trajectory:main',
'hand_tracker = so101_control.hand_tracker:main', # NEW
],
},

Build i test:

cd ~/ros2_ws
colcon build --packages-select so101_control
source install/setup.bash

Uruchom hand tracker

ros2 run so101_control hand_tracker

W drugim terminalu: sprawdź publikowane dane

ros2 topic echo /hand_pose

Powinieneś widzieć aktualizacje Point(x, y, z) gdy poruszasz ręką!

---

7. Mapowanie Trajektorii Ręki na Joint Space Robota

7.1 Strategia Mapowania

Cel: Przekształcić pozycję ręki (wrist Y, middle finger Z) na kąty jointów SO-101.

Mapping scheme:

- **hand_pose.y (wrist vertical position) → joint_1 angle**
  - Range: [0.2, 0.8] (normalized image coordinates)
  - Maps to: [-π/4, π/4] radians (-45° to +45°)
- **hand_pose.z (middle finger base vertical) → joint_2 angle**
  - Range: [0.3, 0.7] (normalized)
  - Maps to: [-π/3, π/3] radians (-60° to +60°)

Funkcja mapowania (linear interpolation):

$$\theta_{joint} = \theta_{min} + \frac{(hand_{pos} - hand_{min})}{(hand_{max} - hand_{min})} \times (\theta_{max} - \theta_{min})$$

7.2 Mapper Node Implementation

Stwórz `so101_control/hand_to_joint_mapper.py`:

#!/usr/bin/env python3

import rclpy
from rclpy.node import Node
from std_msgs.msg import Float64MultiArray
from geometry_msgs.msg import Point
import numpy as np

class HandToJointMapper(Node):
def **init**(self):
super().**init**('hand_to_joint_mapper')

```
    # Subscriber to hand pose
    self.hand_sub = self.create_subscription(
        Point, '/hand_pose', self.hand_callback, 10
    )

    # Publisher to joint commands
    self.joint_pub = self.create_publisher(
        Float64MultiArray, '/position_controller/commands', 10
    )

    # Mapping parameters (tunable!)
    self.declare_parameter('hand_y_min', 0.2)
    self.declare_parameter('hand_y_max', 0.8)
    self.declare_parameter('hand_z_min', 0.3)
    self.declare_parameter('hand_z_max', 0.7)

    self.declare_parameter('joint1_min', -np.pi/4)  # -45 degrees
    self.declare_parameter('joint1_max', np.pi/4)   # +45 degrees
    self.declare_parameter('joint2_min', -np.pi/3)  # -60 degrees
    self.declare_parameter('joint2_max', np.pi/3)   # +60 degrees

    # Smoothing filter (exponential moving average)
    self.alpha = 0.3  # Smoothing factor (0-1, lower = smoother)
    self.prev_joint1 = 0.0
    self.prev_joint2 = 0.0

    self.get_logger().info('Hand-to-Joint Mapper started')
```

```
def hand_callback(self, msg):
    # Get parameters
    hand_y_min = self.get_parameter('hand_y_min').value
    hand_y_max = self.get_parameter('hand_y_max').value
    hand_z_min = self.get_parameter('hand_z_min').value
    hand_z_max = self.get_parameter('hand_z_max').value

    joint1_min = self.get_parameter('joint1_min').value
    joint1_max = self.get_parameter('joint1_max').value
    joint2_min = self.get_parameter('joint2_min').value
    joint2_max = self.get_parameter('joint2_max').value

    # Extract hand coordinates
    hand_y = msg.y  # Wrist vertical position
    hand_z = msg.z  # Middle finger base vertical position

    # Map to joint angles (linear interpolation with clamping)
    joint1_raw = self.map_value(
        hand_y, hand_y_min, hand_y_max, joint1_min, joint1_max
    )
    joint2_raw = self.map_value(
        hand_z, hand_z_min, hand_z_max, joint2_min, joint2_max
    )

    # Apply smoothing filter
    joint1 = self.alpha * joint1_raw + (1 - self.alpha) * self.prev_joint1
    joint2 = self.alpha * joint2_raw + (1 - self.alpha) * self.prev_joint2

    self.prev_joint1 = joint1
    self.prev_joint2 = joint2

    # Create joint command (5 joints: j0, j1, j2, j3, j4)
    joint_cmd = Float64MultiArray()
    joint_cmd.data = [
        0.0,      # joint_0 (base rotation) - not used
        joint1,   # joint_1 (shoulder pitch) - CONTROLLED
        joint2,   # joint_2 (elbow pitch) - CONTROLLED
        0.0,      # joint_3 (wrist pitch) - not used
```

```
        0.0       # joint_4 (wrist roll) - not used
    ]

    # Publish command
    self.joint_pub.publish(joint_cmd)

    # Log for debugging
    self.get_logger().info(
        f'Hand Y: {hand_y:.2f} → J1: {joint1:.2f} | '
        f'Hand Z: {hand_z:.2f} → J2: {joint2:.2f}',
        throttle_duration_sec=0.5
    )

def map_value(self, value, in_min, in_max, out_min, out_max):
    """Linear interpolation with clamping"""
    # Clamp input
    value = max(in_min, min(in_max, value))

    # Map to output range
    mapped = out_min + (value - in_min) / (in_max - in_min) * (out_max - out_

    # Clamp output
    return max(out_min, min(out_max, mapped))
```

def main(args=None):
rclpy.init(args=args)
mapper = HandToJointMapper()

```
try:
    rclpy.spin(mapper)
except KeyboardInterrupt:
    pass
finally:
    mapper.destroy_node()
    rclpy.shutdown()
```

```
if name == 'main':
main()
```

Update `setup.py`:

```
entry_points={
'console_scripts': [
'simple_trajectory = so101_control.simple_trajectory:main',
'hand_tracker = so101_control.hand_tracker:main',
'hand_to_joint_mapper = so101_control.hand_to_joint_mapper:main',
# NEW
],
},
```

---

8. Przeniesienie Ruchu do Symulacji Gazebo

8.1 Integrated Launch File

Stwórz `so101_control/launch/hand_control.launch.py`:

```
import os
from ament_index_python.packages import
get_package_share_directory
from launch import LaunchDescription
from launch_ros.actions import Node
from launch.actions import IncludeLaunchDescription
from launch.launch_description_sources import
PythonLaunchDescriptionSource

def generate_launch_description():
# Include Gazebo with SO-101
gazebo_launch = IncludeLaunchDescription(
PythonLaunchDescriptionSource([
os.path.join(
get_package_share_directory('so101_description'),
'launch',
'gazebo_so101.launch.py'
)
])
)
```

```
# Hand tracker node
hand_tracker = Node(
    package='so101_control',
    executable='hand_tracker',
    output='screen'
)

# Hand-to-joint mapper node
mapper = Node(
    package='so101_control',
    executable='hand_to_joint_mapper',
    output='screen',
    parameters=[{
        'hand_y_min': 0.2,
        'hand_y_max': 0.8,
        'hand_z_min': 0.3,
        'hand_z_max': 0.7,
        'joint1_min': -0.785,  # -45 degrees
        'joint1_max': 0.785,   # +45 degrees
        'joint2_min': -1.047,  # -60 degrees
        'joint2_max': 1.047    # +60 degrees
    }]
)

return LaunchDescription([
    gazebo_launch,
    hand_tracker,
    mapper
])
```

Dodaj launch do `setup.py`:

```
import os
from glob import glob

setup(
# ... existing code ...
data_files=[
```

```
('share/ament_index/resource_index/packages',
['resource/' + package_name]),
('share/' + package_name, ['package.xml']),
(os.path.join('share', package_name, 'launch'),
glob('launch/*.launch.py')), # NEW
],
# ... rest of setup ...
)
```

Stwórz katalog launch:

```
cd ~/ros2_ws/src/so101_control
mkdir launch
```

Przenieś hand_control.launch.py do launch/

8.2 Complete System Test

Build wszystko:

```
cd ~/ros2_ws
colcon build
source install/setup.bash
```

Uruchom pełny system:

```
ros2 launch so101_control hand_control.launch.py
```

Co się dzieje:

1. Gazebo uruchamia się z modelem SO-101
2. Hand tracker zaczyna przechwytywać obraz z kamery
3. Mapper przekształca pozycję ręki na joint angles
4. SO-101 w Gazebo naśladuje ruch Twojej ręki w real-time!

Pokaż rękę przed kamerą i poruszaj nią w pionie - ramię SO-101 powinno się poruszać synchronicznie!

---

9. Testing i Debugging

9.1 Common Issues i Rozwiązania

Problem 1: Kamera nie wykrywa ręki

- **Przyczyna:** Złe oświetlenie, zbyt szybki ruch
- **Rozwiązanie:**
  - Zwiększ oświetlenie (światło naturalne lub lampka)
  - Zmniejsz `min_detection_confidence` do 0.5
  - Poruszaj ręką wolniej

Problem 2: Robot się trzęsie (jittery motion)

- **Przyczyna:** Zbyt mały smoothing factor
- **Rozwiązanie:**
  - Zwiększ `alpha` w mapper (np. z 0.3 do 0.5)
  - Dodaj dodatkowy low-pass filter

Problem 3: Joint limity są przekroczone

- **Przyczyna:** Nieprawidłowe mapowanie zakresu
- **Rozwiązanie:**
  - Dostosuj parametry `joint1_min/max` w launch file
  - Sprawdź faktyczne joint limits w URDF

Problem 4: Lag między ręką a robotem

- **Przyczyna:** Zbyt wolny controller update rate
- **Rozwiązanie:**
  - Zwiększ `update_rate` w `so101_controllers.yaml` (np. 200 Hz)
  - Zmniejsz `alpha` (więcej responsiveness, mniej smoothing)

9.2 Debugging Tools

RViz dla wizualizacji TF frames:

W nowym terminalu

ros2 run rviz2 rviz2

Dodaj displays:

- RobotModel (topic: /robot_description)

- TF

- Camera (topic: /camera/image_raw) - jeśli dostępny

rqt_graph do sprawdzenia node connections:

ros2 run rqt_graph rqt_graph

Powinieneś zobaczyć:

hand_tracker → /hand_pose → hand_to_joint_mapper → /position_controller/commands → Gazebo

Monitoring joint states:

Real-time joint positions

ros2 topic echo /joint_states

Hand pose coordinates

ros2 topic echo /hand_pose

Joint commands being sent

ros2 topic echo /position_controller/commands

9.3 Performance Tuning

Parametry do eksperymentowania:

Parameter	Default	Effect
alpha (smoothing)	0.3	Lower = smoother, higher = responsive
hand_y_min	0.2	Adjust based on camera FOV
hand_y_max	0.8	Adjust based on camera FOV
joint1_min	-45°	Shoulder range limit
joint1_max	+45°	Shoulder range limit
controller update_rate	100 Hz	Higher = less lag, more CPU

Table 3: Tunable parameters dla performance optimization

Zmień parametry w runtime:

Change mapping range

ros2 param set /hand_to_joint_mapper hand_y_min 0.15
ros2 param set /hand_to_joint_mapper hand_y_max 0.85

---

10. Następne Kroki: Shape Sorting Project

10.1 Od Hand Mimicking do Shape Recognition

Ten projekt dał Ci:

- ✅ SO-101 model w Gazebo
- ✅ ROS 2 control pipeline
- ✅ Computer vision integration (MediaPipe)
- ✅ Coordinate mapping (image space → joint space)
- ✅ Real-time teleoperation

Shape sorting project będzie rozszerzeniem:

1. **Vision:** MediaPipe Hand → **OpenCV Shape Detection**
2. **Recognition:** Hand landmarks → **Shape classification (circle, square, triangle)**
3. **Motion:** Simple mapping → **Pick-and-place trajectory planning**
4. **End-effector:** None → **Gripper design + grasp control**

10.2 Shape Sorting Architecture Preview

Pipeline:

Figure 2: Shape sorting project architecture

1. **Camera captures scene** with blocks (circle, square, triangle)
2. **OpenCV detects shapes** using contour detection + Hu moments
3. **Shape classifier** determines which block to pick
4. **Pose estimator** calculates 6D pose (position + orientation)

5. **Motion planner** generates pick trajectory (approach → grasp → lift)
6. **SO-101 executes** pick-and-place to matching bin
7. **Repeat** until all blocks sorted

10.3 Required Extensions

Co musisz dodać:

Component	Technology	Complexity
Gripper model	URDF + Gazebo grasp plugin	⭑⭑
Shape detection	OpenCV (contours, moments)	⭑⭑
Shape classification	SVM or simple neural network	⭑⭑
Pick-and-place planner	MoveIt 2 Cartesian paths	⭑⭑⭑
Grasp pose calculation	PnP algorithm + transforms	⭑⭑⭑
Bin localization	ArUco markers or fixed poses	⭑

Table 4: Shape sorting project components

Szacowany czas: 40-50 godzin (4-5 tygodni)

10.4 Recommended Learning Path

Przed rozpoczęciem shape sorting:

1. **Ukończ ten projekt** (hand mimicking) - 35-45h
2. **Dodaj gripper do SO-101** - 10-15h
   - Design simple 1-DOF gripper w URDF
   - Gazebo grasp plugin configuration
   - ROS 2 gripper controller
3. **OpenCV shape detection tutorial** - 5-10h
   - Contour detection
   - Shape classification (Hu moments)
   - Real-time detection w ROS 2
4. **Basic pick-and-place** - 15-20h

    - Fixed-pose pick-and-place (no vision)
    - MoveIt 2 Cartesian planning
    - Grasp/release logic
5. **Integration** - 10-15h
    - Vision-guided grasping
    - Multi-shape sorting loop
    - Error handling

Total: ~75-115h (2-3 miesiące part-time)

10.5 Zasoby dla Shape Sorting

Tutorials:

- OpenCV Shape Detection: https://docs.opencv.org/4.x/d0/d49/tutorial_moments.html
- MoveIt 2 Pick-and-Place: https://moveit.picknik.ai/main/doc/examples/pick_place/pick_place_tutorial.html
- Gazebo Grasp Plugin: https://github.com/JenniferBuehler/gazebo-pkgs/wiki/The-Gazebo-grasp-fix-plugin

Example projects:

- Shape Sorting with UR5: https://github.com/cambel/ur5_shape_sorting
- ROS 2 Pick-and-Place Demo: https://github.com/ros-planning/moveit2_tutorials

---

11. Podsumowanie i Checklist

11.1 Co Osiągnąłeś w Tym Projekcie

Technical skills:

- ✅ Import CAD modelu do ROS 2/Gazebo
- ✅ URDF/Xacro modeling dla manipulatora
- ✅ Gazebo physics simulation + ROS 2 control
- ✅ Computer vision pipeline (MediaPipe + OpenCV)
- ✅ Coordinate transformation (image → joint space)
- ✅ Real-time robot teleoperation
- ✅ ROS 2 node architecture (publisher/subscriber pattern)

- ✅ Parameter tuning i debugging

Koncepcyjne:

- ✅ Vision-based control fundamentals
- ✅ Joint space vs task space mapping
- ✅ Robot kinematics basics
- ✅ Sensor-to-actuator pipeline

11.2 Complete Project Checklist

Setup (Week 1):

☐ Zainstaluj Ubuntu 24.04
☐ Zainstaluj ROS 2 Jazzy
☐ Zainstaluj Gazebo Harmonic
☐ Zainstaluj OpenCV + MediaPipe
☐ Stwórz ROS 2 workspace
☐ Sklonuj SO-100 repo
☐ Stwórz so101_description package
☐ Zaimportuj URDF + meshes
☐ Uruchom SO-101 w Gazebo (pierwszy test!)

Control (Week 2):

☐ Napisz simple_trajectory.py
☐ Przetestuj manual joint control via CLI
☐ Zweryfikuj robot response w Gazebo
☐ Stwórz so101_control package
☐ Implementuj hand_tracker.py
☐ Przetestuj MediaPipe detection standalone
☐ Zintegruj hand tracker z ROS 2
☐ Verify /hand_pose topic publikuje dane

Mapping (Week 3):

☐ Implementuj hand_to_joint_mapper.py
☐ Skonfiguruj mapowanie parametrów
☐ Przetestuj mapowanie w isolation
☐ Dodaj smoothing filter
☐ Tune mapping ranges
☐ Stwórz integrated launch file

☐ Test pełnego systemu (hand → Gazebo)
☐ Fine-tune responsiveness vs smoothness

Testing & Documentation (Week 4):

☐ Test różnych oświetleń
☐ Test różnych pozycji ręki
☐ Measure latency (hand motion → robot motion)
☐ Zapisz video demo
☐ Napisz README.md
☐ Dokumentuj parametry tuning
☐ Push na GitHub
☐ Planuj shape sorting project

11.3 GitHub Repository Structure

Rekomendowana struktura dla Twojego projektu:

```
so101_hand_mimicking/
├── README.md
├── docs/
│   ├── setup_guide.md
│   ├── troubleshooting.md
│   └── images/
│       ├── architecture.png
│       ├── gazebo_screenshot.png
│       └── hand_tracking_demo.gif
├── so101_description/
│   ├── CMakeLists.txt
│   ├── package.xml
│   ├── urdf/
│   ├── meshes/
│   ├── launch/
│   └── config/
├── so101_control/
│   ├── setup.py
│   ├── package.xml
│   ├── so101_control/
│   │   ├── __init__.py
│   │   ├── hand_tracker.py
│   │   ├── hand_to_joint_mapper.py
```

```
│ │ └── simple_trajectory.py
│ └── launch/
│ └── hand_control.launch.py
└── scripts/
├── install_dependencies.sh
└── test_hand_tracking.py
```

11.4 Next Project Preparation

Przed rozpoczęciem shape sorting, przygotuj:

1. **3D-printed gripper** (opcjonalnie fizyczny)
    - Download gripper STL z Thingiverse/GrabCAD
    - Import do so101_description/meshes
2. **Shape blocks models**
    - Stwórz simple geometry w Gazebo (box, cylinder, prism)
    - Export jako SDF models
3. **OpenCV learning**
    - Complete OpenCV shape detection tutorial
    - Experiment z real objects
4. **MoveIt 2 basics**
    - MoveIt Setup Assistant dla SO-101
    - Basic motion planning examples

---

12. Resources i Community

12.1 Official Documentation

- **ROS 2 Jazzy:** https://docs.ros.org/en/jazzy/
- **Gazebo Harmonic:** https://gazebosim.org/docs/harmonic
- **MediaPipe:** https://developers.google.com/mediapipe
- **OpenCV:** https://docs.opencv.org/4.x/
- **LeRobot:** https://huggingface.co/docs/lerobot

12.2 Community Support

- **ROS Discourse:** https://discourse.ros.org/
- **Gazebo Answers:** https://answers.gazebosim.org/
- **r/ROS (Reddit):** https://reddit.com/r/ROS
- **Stack Overflow:** Tags: ros2, gazebo, opencv

12.3 Similar Projects (Inspiracja)

- **myCobot YOLOv8 Hand Mimicking:** https://www.hackster.io/Elephant-Robotics-Official/yolov8-mycobot-mimicking-human-movement-402f92
- **Hand Gesture ROS2 Controller:** https://github.com/ratulKabir/ros2_dual_arm_control_by_gesture
- **SO-100 ROS 2 Package:** https://github.com/brukg/SO-100-arm

---

13. Appendix: Quick Commands Reference

13.1 Setup Commands

ROS 2 setup

```
source /opt/ros/jazzy/setup.bash
source ~/ros2_ws/install/setup.bash
```

Build workspace

```
cd ~/ros2_ws
colcon build
colcon build --packages-select so101_description so101_control
```

Clean build

```
rm -rf build install log
colcon build
```

13.2 Launch Commands

Launch Gazebo with SO-101

```
ros2 launch so101_description gazebo_so101.launch.py
```

Launch hand tracking only

ros2 run so101_control hand_tracker

Launch mapper only

ros2 run so101_control hand_to_joint_mapper

Launch full system

ros2 launch so101_control hand_control.launch.py

13.3 Debugging Commands

List all nodes

ros2 node list

List all topics

ros2 topic list

Echo topic data

ros2 topic echo /hand_pose
ros2 topic echo /joint_states
ros2 topic echo /position_controller/commands

Check node info

ros2 node info /hand_tracker
ros2 node info /hand_to_joint_mapper

View TF tree

ros2 run tf2_tools view_frames

Launch RViz

ros2 run rviz2 rviz2

Check parameters

ros2 param list /hand_to_joint_mapper
ros2 param get /hand_to_joint_mapper hand_y_min
ros2 param set /hand_to_joint_mapper hand_y_min 0.15

13.4 Testing Commands

Manual joint control

ros2 topic pub --once /position_controller/commands
std_msgs/msg/Float64MultiArray
"{data: [0.0, 0.5, -0.3, 0.0, 0.0]}"

Run simple trajectory

ros2 run so101_control simple_trajectory

Test hand tracking standalone

python3 test_hand_tracking.py

🎉 Gratulacje!

Ukończyłeś kompletny przewodnik dla projektu SO-101 Hand Mimicking!

Ten dokument daje Ci:

- ✅ Pełną ścieżkę od zera do działającego systemu
- ✅ Wszystkie niezbędne komendy i kod
- ✅ Teoretyczne podstawy i praktyczne implementacje
- ✅ Debugging strategies i tuning guidelines
- ✅ Przygotowanie do shape sorting project

Powodzenia w realizacji projektu! 🚀

Jeśli masz pytania podczas implementacji, wróć do sekcji 9 (Testing & Debugging) lub zapytaj na ROS Discourse.

Happy coding! 🤖

References

[1] Hugging Face LeRobot Team. (2024). SO-100 Robot Arm Documentation. *Hugging Face LeRobot*. https://huggingface.co/docs/lerobot/so100

[2] Open Robotics. (2024). ROS 2 Jazzy Documentation. *ROS 2 Docs*. https://docs.ros.org/en/jazzy/

[3] Open Robotics. (2024). Gazebo Harmonic Documentation. *Gazebo Sim*. https://gazebosim.org/docs/harmonic

[4] Google. (2024). MediaPipe Hands Solution. *Google Developers*. https://developers.google.com/mediapipe/solutions/vision/hand_landmarker

[5] OpenCV Team. (2024). OpenCV 4.x Documentation. *OpenCV Docs*. https://docs.opencv.org/4.x/

[6] Elephant Robotics. (2024). YOLOv8 myCobot Mimicking Human Movement. *Hackster.io*. https://www.hackster.io/Elephant-Robotics-Of

ficial/yolov8-mycobot-mimicking-human-movement-402f92

[7] Kabir, R. (2025). ROS2 Dual Arm Control by Gesture. *GitHub Repository*. https://github.com/ratulKabir/ros2_dual_arm_control_by_gesture

[8] Brukg. (2024). SO-100 Robot Arm ROS2 Package. *GitHub Repository*. https://github.com/brukg/SO-100-arm